Universidad Tecnológica de Honduras

ASIGNATURA:
PROGRAMACION MOVIL I

TRABAJO:
Tarea 2.2

CATEDRÁTICO:
Ricardo Lagos

REALIZADO POR:

José Fernando Orellana Reyes (202210010352)
Carlos Roberto Cardenas Martínez (202430010599)
Jennifer Mariela Marquez Chacon (202330010411)
Michael Abad Diaz Castro (202310040165)
Fernando Abad Diaz Castro (202410040257)
Romel Donaldo Barahona Márquez (202410010359)

REALIZADO POR:
Repositorio: github.com/Kirklen0910/Exa1Pa1199

FECHA:
07 noviembre de 2026

CONTENIDO

1. Introducción
2. Arquitectura por capas
3. Entidad Registro y reglas del negocio
4. Endpoints de la API
5. Validación y manejo de errores
6. Documentación OpenAPI
7. Pruebas con Postman
8. Evidencias de las pruebas
9. Repositorio y ejecución
10. Distribución del trabajo
11. Conclusiones

1. INTRODUCCIÓN

En el primer parcial cada equipo construyó una aplicación que guardaba en SQLite registros con fotografía y una descripción obtenida por voz. En el segundo parcial esa información pasa a un servidor. En esta tarea se desarrolló la API REST que recibe y administra esos registros, llamada Bitácora de campo, y que será consumida por la aplicación Android en las tareas 2.3, 2.4 y 2.5.

La API se construyó en C# con ASP.NET Core, organizada en capas según los principios de arquitectura limpia, con acceso a datos mediante Entity Framework Core y SQLite. Se documentó con OpenAPI (Swagger) y se probó con una colección de Postman.

2. ARQUITECTURA POR CAPAS

La solución Bitacora.sln contiene cuatro proyectos. Las dependencias apuntan hacia el dominio: la capa de presentación conoce a la aplicación y a la infraestructura, mientras que el dominio no conoce a ninguna otra capa (Figura 1).

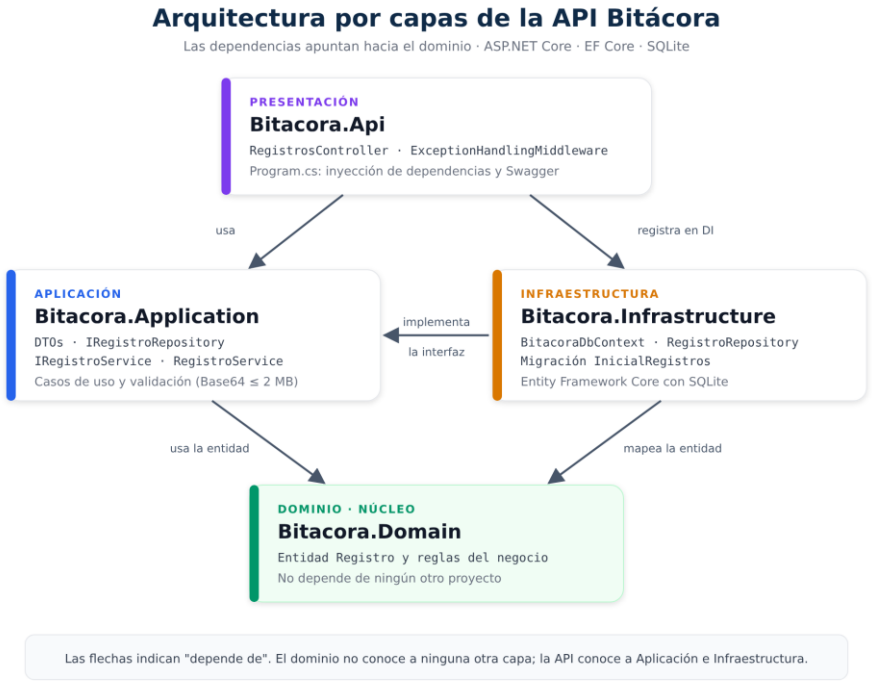


Figura 1. Diagrama de capas y dependencias de la solución

Tabla 1. Proyectos de la solución

Proyecto	Contenido	Depende de
Bitacora.Domain	Entidad Registro y reglas del negocio	Ninguno
Bitacora.Application	DTOs, IRegistroRepository, IRegistroService, RegistroService y validación Base64	Domain

Proyecto	Contenido	Depende de
Bitacora.Infrastructure	BitacoraDbContext, RegistroRepository y migración InicialRegistros (EF Core + SQLite)	Application, Domain
Bitacora.Api	RegistrosController, middleware de excepciones, inyección de dependencias y Swagger	Application, Infrastructure

3. ENTIDAD REGISTRO Y REGLAS DEL NEGOCIO

La entidad Registro representa una entrada de la bitácora. Sus límites se definen como constantes en el dominio y se reutilizan en la validación de los DTO y en la configuración de Entity Framework Core.

Tabla 2. Campos de la entidad Registro

Campo	Tipo	Regla
Id	int	Identificador asignado por la base de datos
Titulo	string	Obligatorio, entre 3 y 100 caracteres
Descripcion	string	Obligatoria, máximo 500 caracteres (en Android se obtiene por voz)
FotoBase64	string (opcional)	Si se envía, debe ser Base64 válido y no superar 2 MB
FechaCreacion	DateTime	La asigna el servidor en UTC; el cliente no la envía

4. ENDPOINTS DE LA API

El controlador RegistrosController expone las cinco operaciones CRUD. Nunca devuelve la entidad de base de datos: utiliza DTO de salida.

Tabla 3. Endpoints y códigos de respuesta

Método	Ruta	Acción	Respuestas
GET	/api/registros	Listar registros sin la foto completa (indica tieneFoto)	200
GET	/api/registros/{id}	Consultar un registro con su foto	200, 404
POST	/api/registros	Crear un registro	201 con Location, 400
PUT	/api/registros/{id}	Actualizar un registro	204, 400, 404
DELETE	/api/registros/{id}	Eliminar un registro	204, 404

5. VALIDACIÓN Y MANEJO DE ERRORES

- **Validación de entrada:** los DTO usan Data Annotations y un atributo propio (Base64ImagenAttribute) que comprueba que la foto sea Base64 válido y no supere 2 MB. Si hay errores, la API responde 400 con un ValidationProblemDetails que detalla cada campo inválido.
- **Recurso inexistente:** las operaciones por id responden 404 con un objeto ProblemDetails.
- **Errores internos:** ExceptionHandlingMiddleware captura cualquier excepción no controlada y responde 500 con un ProblemDetails genérico, sin exponer el detalle interno del error.
- **Negociación de contenido:** si el cliente solicita un formato no soportado (por ejemplo, XML), la API responde 406 Not Acceptable.

6. DOCUMENTACIÓN OPENAPI

La API habilita OpenAPI y muestra su documentación con Swagger UI en http://localhost:5080/swagger, donde se pueden consultar y ejecutar los cinco endpoints.

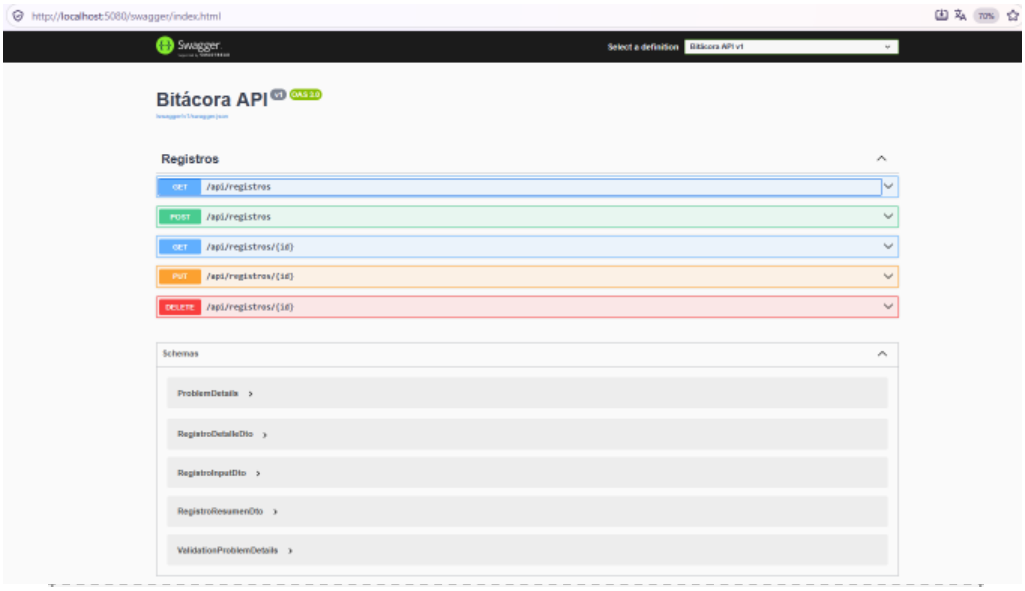


Figura 2. Documentación de la API en Swagger UI

7. PRUEBAS CON POSTMAN

Se creó la colección "Bitácora API" con el entorno "Bitácora - Local", que define la variable baseUrl = http://localhost:5080. Cada endpoint tiene al menos una prueba exitosa y una fallida, con scripts que verifican el código de estado. El POST exitoso guarda el id creado en una variable para usarlo en las siguientes peticiones.

Tabla 4. Resultados de las pruebas en Postman

N.º	Prueba	Respuesta	Resultado
1	Crear registro con foto	201 Created	Aprobada

N.º	Prueba	Respuesta	Resultado
2	Crear registro sin foto	201 Created	Aprobada
3	Crear registro inválido	400 Bad Request	Aprobada
4	Listar registros	200 OK	Aprobada
5	Listar pidiendo XML	406 Not Acceptable	Aprobada
6	Obtener registro existente	200 OK	Aprobada
7	Obtener registro inexistente	404 Not Found	Aprobada
8	Actualizar registro	204 No Content	Aprobada
9	Actualizar con datos inválidos	400 Bad Request	Aprobada
10	Actualizar registro inexistente	404 Not Found	Aprobada
11	Eliminar registro	204 No Content	Aprobada
12	Eliminar registro sin foto	204 No Content	Aprobada
13	Eliminar registro ya eliminado	404 Not Found	Aprobada

8. EVIDENCIAS DE LAS PRUEBAS

8.1 POST /api/registros

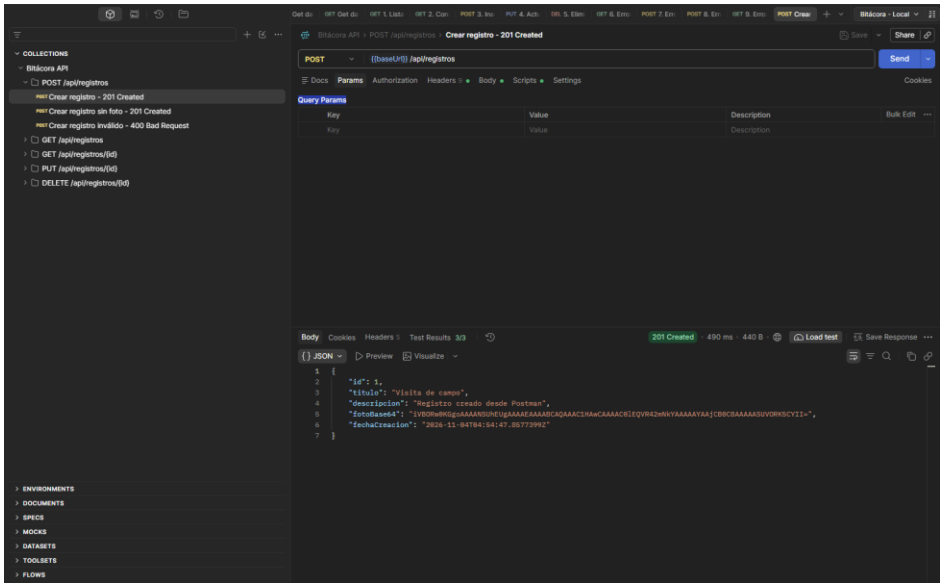


Figura 3. Crear registro con foto: 201 Created

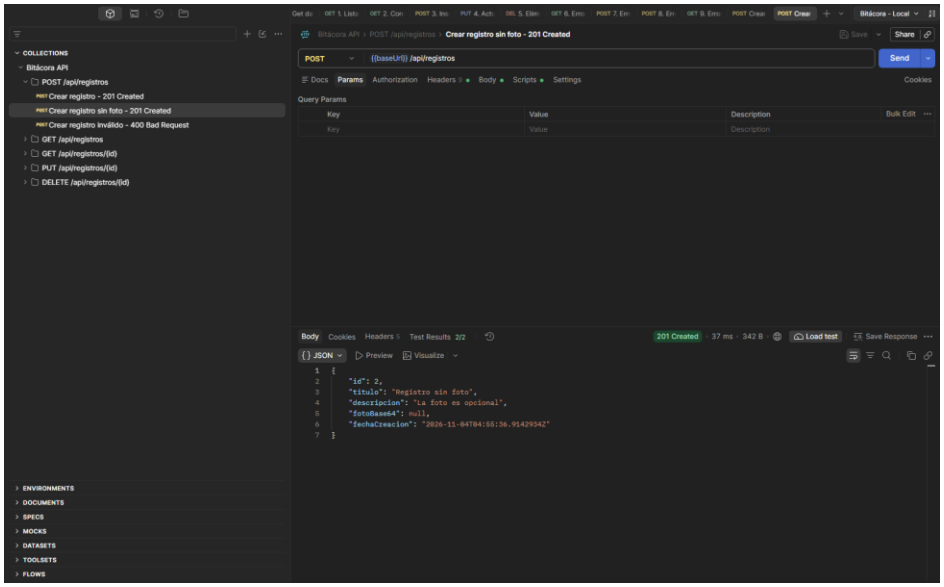


Figura 4. Crear registro sin foto: 201 Created, fotoBase64 es null

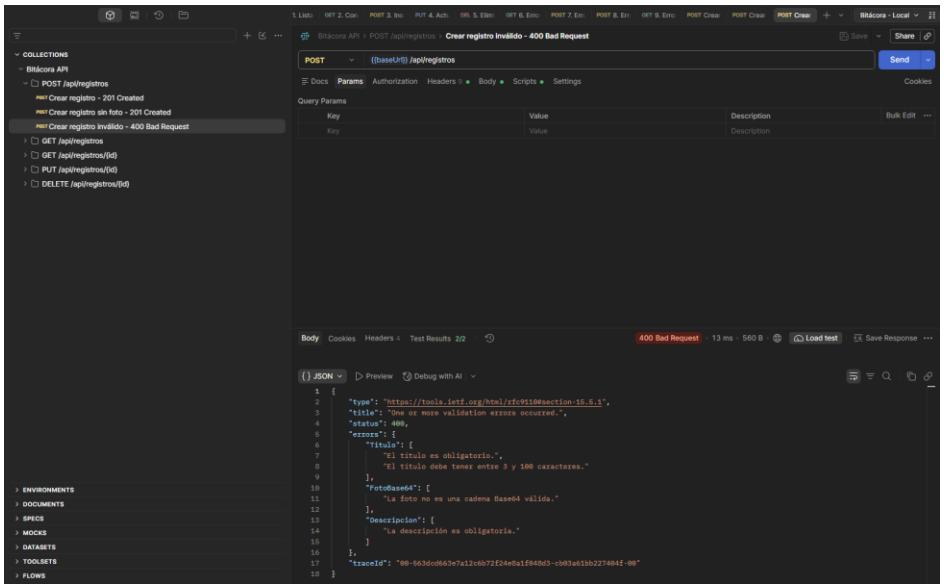


Figura 5. Crear registro inválido: 400 con el detalle de cada campo

8.2 GET /api/registros

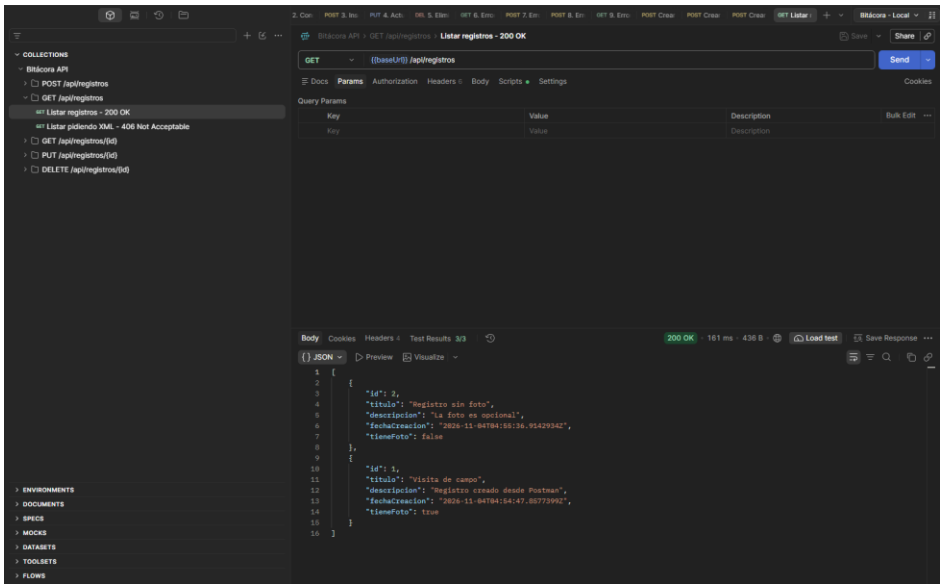


Figura 6. Listar registros: 200 OK, sin la foto completa

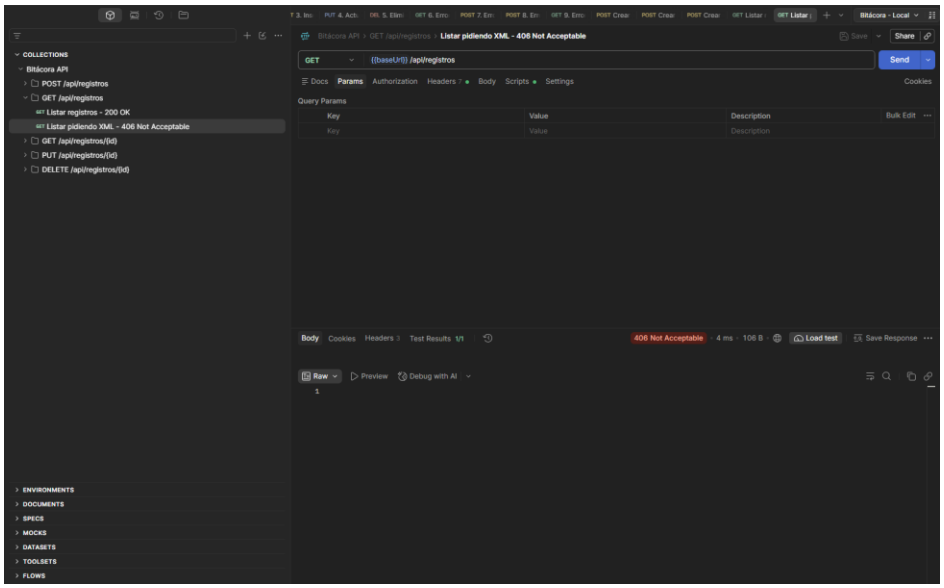


Figura 7. Listar pidiendo XML: 406 Not Acceptable

8.3 GET /api/registros/{id}

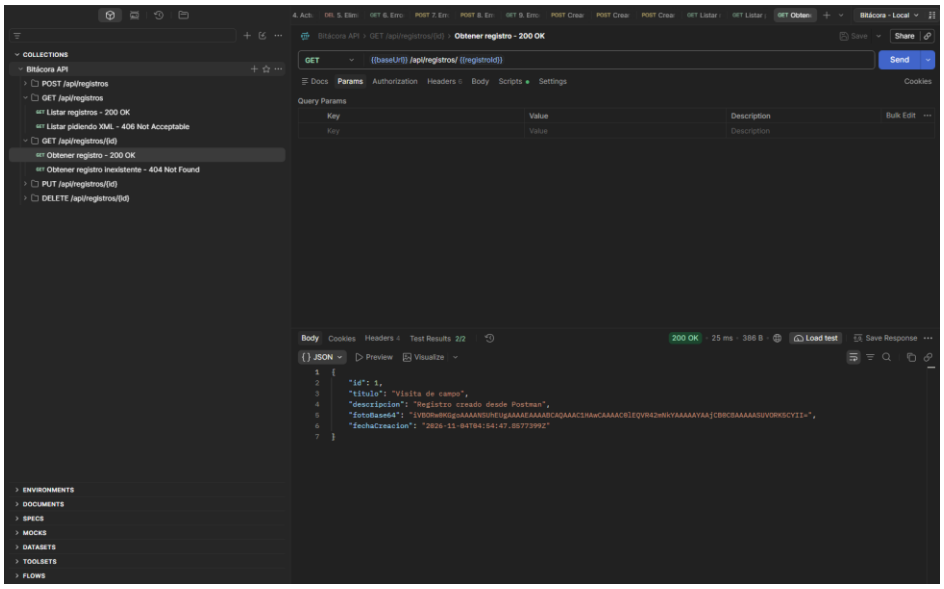


Figura 8. Obtener registro: 200 OK con la foto

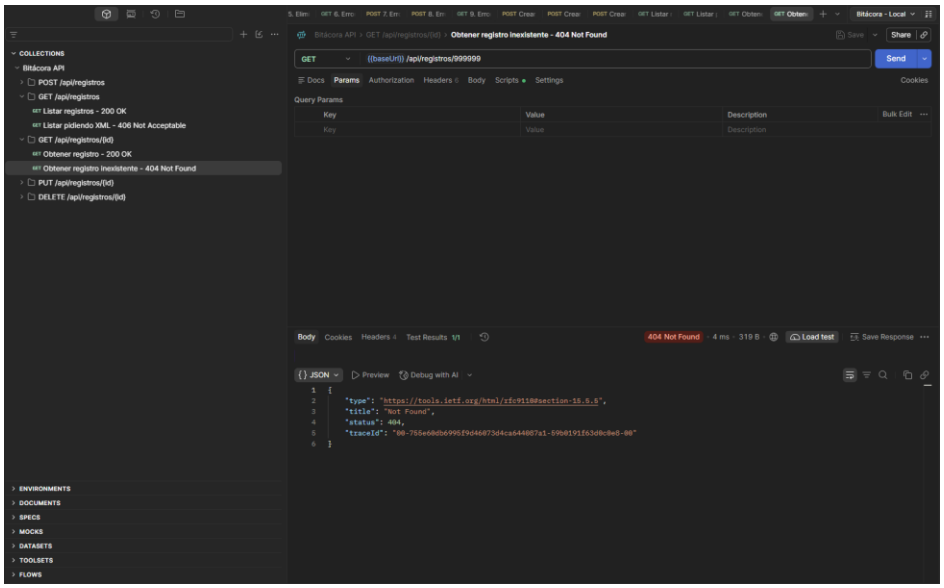


Figura 9. Obtener registro inexistente: 404 Not Found

8.4 PUT /api/registros/{id}

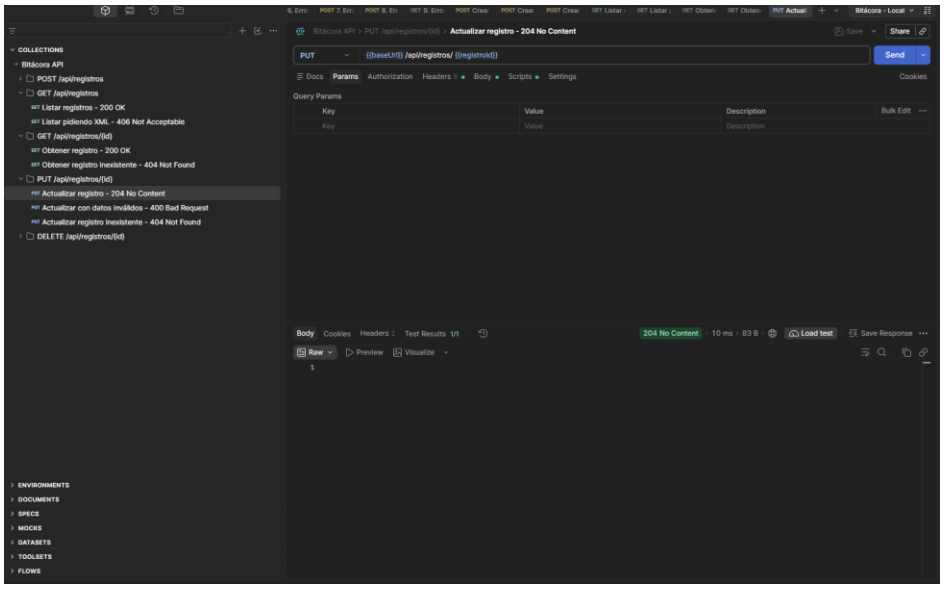


Figura 10. Actualizar registro: 204 No Content

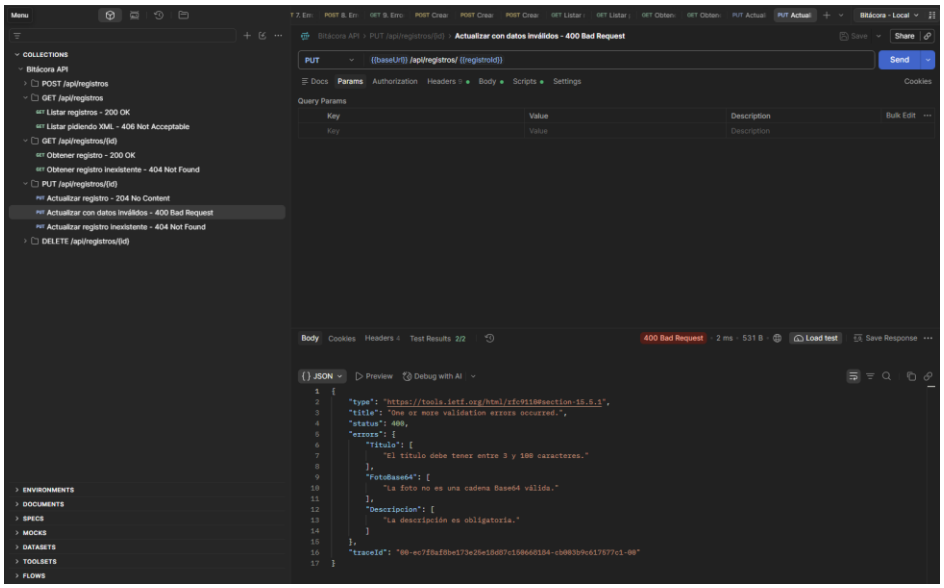


Figura 11. Actualizar con datos inválidos: 400 Bad Request

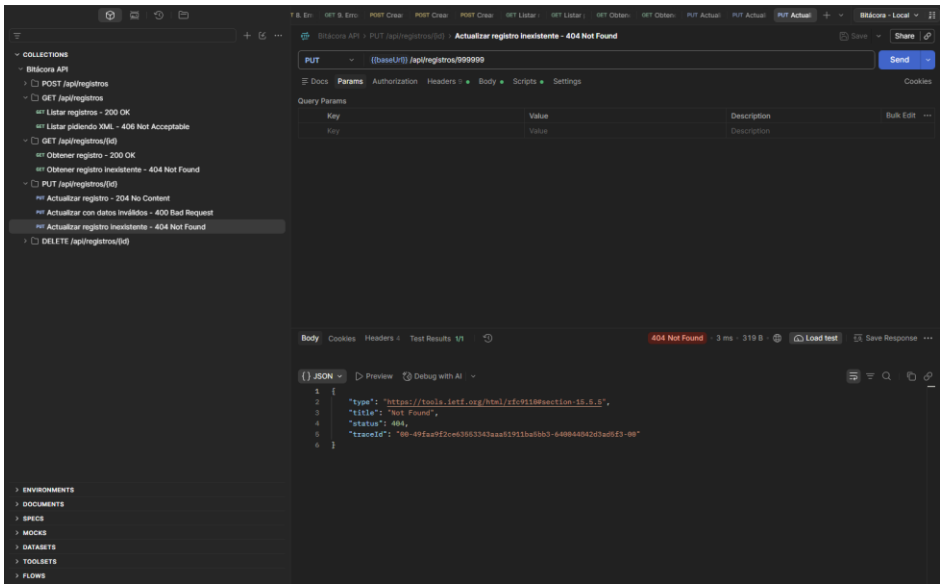


Figura 12. Actualizar registro inexistente: 404 Not Found

8.5 DELETE /api/registros/{id}

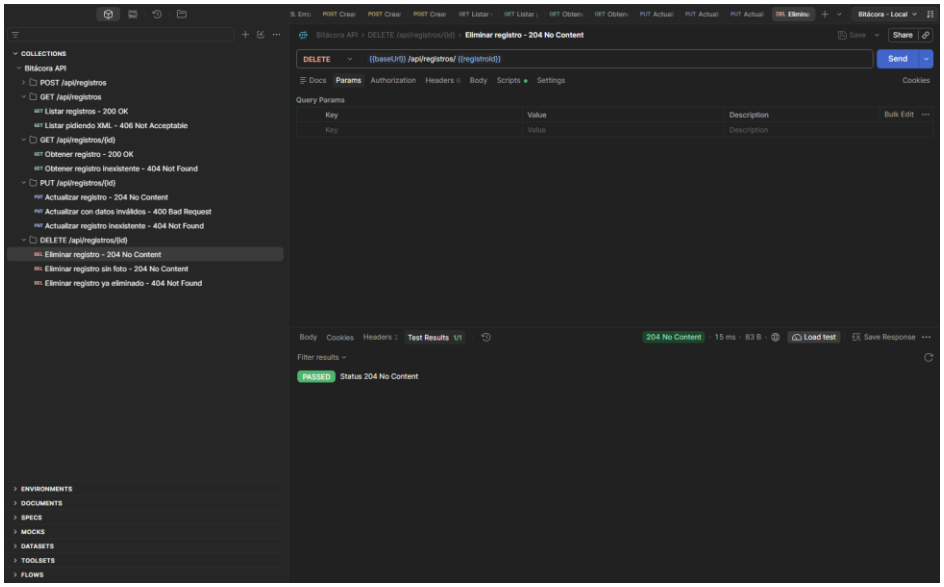


Figura 13. Eliminar registro: 204 No Content

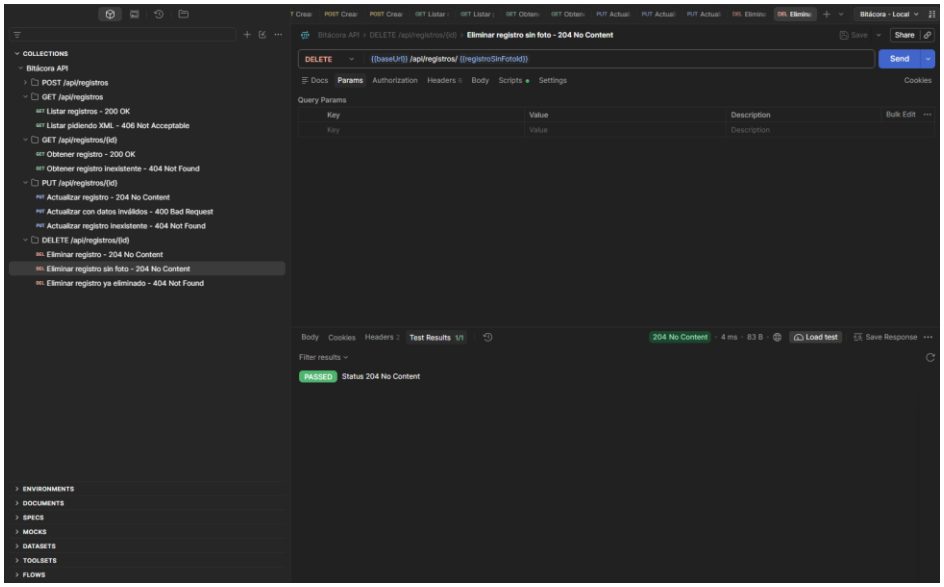


Figura 14. Eliminar registro sin foto: 204 No Content

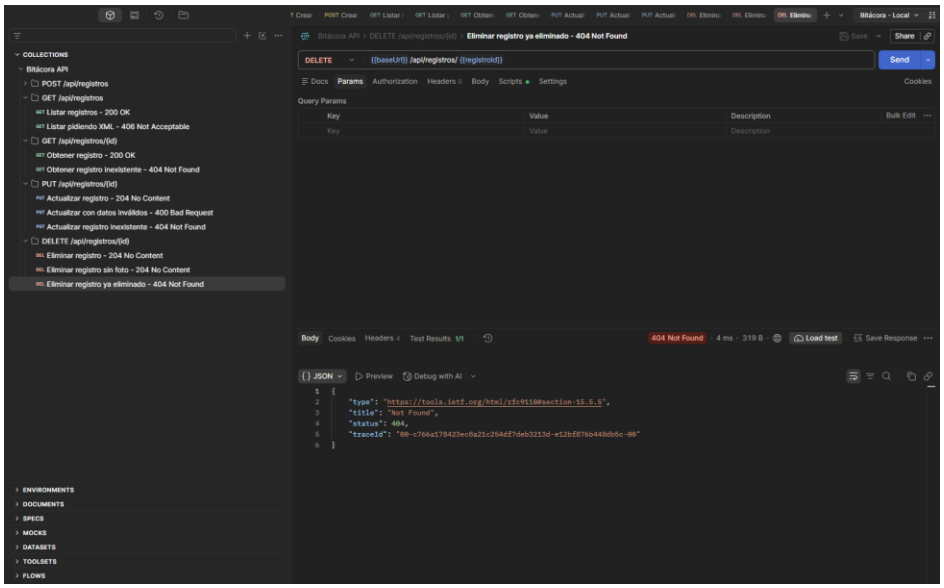


Figura 15. Eliminar registro ya eliminado: 404 Not Found

9. REPOSITORIO Y EJECUCIÓN

- **Repositorio:** github.com/Kirklen0910/Exa1Pa1199, carpeta BitacoraApi.
- **.gitignore:** excluye las carpetas bin y obj y los archivos de base de datos.
- **README:** BitacoraApi/README.md describe cómo ejecutar la API y aplicar las migraciones.

Pasos para ejecutar la API:

- cd BitacoraApi/src/Bitacora.Api
- dotnet ef database update --project ../Bitacora.Infrastructure
- dotnet run

10. DISTRIBUCIÓN DEL TRABAJO

Cada integrante desarrolló una capa en su propia rama y la integró a main mediante un Pull Request. El orden de integración siguió las dependencias, desde el dominio hasta la presentación.

Tabla 5. Distribución del trabajo

Integrante	Rama	PR	Aporte
Carlos Roberto Cárdenas Martínez	main, fix/application	#5	Estructura de la solución, corrección de la capa Application, revisión e integración de los Pull Requests
Michael Abad Díaz Castro	feature/domain	#1	Capa Domain: entidad Registro y reglas del negocio
Romel Márquez	feature/application	#2	Capa Application: DTOs, validación y servicio
José Fernando Orellana Reyes	feature/infrastructure	#3	Capa Infrastructure: EF Core, SQLite, repositorio y migración

Integrante	Rama	PR	Aporte
Jennifer Mariela Márquez Chacón	feature/api	#4	Capa Api: controlador, middleware de errores y Swagger
Fernando Abad	docs/postman-readme	#6	Colección y entorno de Postman, README y documentación

11. CONCLUSIONES

- La separación en cuatro proyectos con dependencias hacia el dominio mantiene las reglas del negocio aisladas de la base de datos y del framework web.
- Los DTO y la validación centralizada garantizan respuestas 400 claras y evitan exponer la entidad de base de datos.
- El middleware de excepciones y ProblemDetails dan un formato uniforme a todos los errores sin revelar información interna.
- Swagger y la colección de Postman permitieron documentar y verificar los cinco endpoints, tanto en los casos exitosos como en los de error.
- La API queda lista y documentada como base para las tareas 2.3, 2.4 y 2.5.